



FREE FIELD GUIDE

HOMELAB STARTER

Your First Build That Gets You Hired

BUILD IT. BREAK IT. TALK ABOUT IT.

A free guide from James at Twilight Tech. Build it. Break it. Talk about it.

Before we start — a confession

I'm going to tell you the truth right out of the gate, because I'd rather you trust me than think I'm selling you something.

This guide does **not** teach you the A+. It's not a course. I'm not going to explain what DNS *is* from scratch or walk you through the seven layers of the OSI model. There's already a guy who does that better than almost anyone, and he does it **for free: Professor Messer**. Go watch his A+ and Network+ playlists on YouTube. Seriously. If you don't know the vocabulary yet, start there, come back here, and you'll get ten times more out of this.

So what *is* this?

This is the thing nobody handed me when I was studying. This is me, a few steps up the trail, turning around and saying: *here's the build that actually got me hired, here's exactly how to do it, and here's how to talk about it so an interviewer believes you.*

Because here's what they don't tell you when you're grinding flashcards: **the certificate is the ticket in. The story is what gets you the job.**

Let me prove it.

The night that actually got me hired

I'll skip ahead to an interview I had. They asked me the classic question – *"Tell me about a time you fixed something."*

And I want you to know: I had studied. I had done A+ practice questions until my eyes crossed. I'd seen hundreds of them. And in that moment, with a real human looking at me waiting for an answer, **not one single practice question came to mind**. Nothing. The whole bank of memorized trivia just... evaporated.

You know what *did* come out?

The night I sat in front of my homelab at **midnight**, exhausted, trying to figure out why the hell a workstation **would not join the domain**. I'd done everything right. Server was up. Domain was built. User account existed. And this one client machine just kept throwing errors and refusing to join, over and over, and I could not figure out why.

I walked the interviewer through the whole thing. Every dead end. The frustration. And finally the moment I found it: the workstation's **IP address wasn't even on the network**. It was sitting on the wrong subnet. It literally could not *reach* the domain controller to talk to it – it was shouting into a room the DC wasn't in. Fixed the IP, put it on the right network, and it joined in five seconds.

I told that story with **only A+ Core 1 done**. Not even the full A+.

And I got the job.

Not because I had a cert. Because I'd actually **been there at midnight**, hit a real wall, and clawed my way through it – and I could *talk about it* like someone who'd lived it. Because I had.

That's the whole method. That's everything I'm going to teach you:

The build in this guide is the *exact* kind of build that gave me that story. Let's go make yours.

What you'll build

You're going to build the **simplest version of a real enterprise network**. Not a toy. The real thing, just small.

Two machines:

1. **A Windows Server acting as a Domain Controller (DC)**. This is the brain of a company network – it holds the user accounts, decides who's allowed in, and runs DNS so machines can find each other. Every office you've ever badged into has one of these (usually several) humming in a closet somewhere.
2. **A Windows client workstation that *joins* that domain**. This is the regular employee PC. When it "joins the domain," it stops being a lonely island and becomes part of the company – it trusts the server, it lets domain users log in, it follows the rules the server hands down.

That handshake – a client joining a domain controller – is the **single most fundamental thing** in Windows enterprise IT. Help desk, sysadmin, desktop support – they all run on this. If you understand this cold, you understand the bones of the job.

What you need

- **One computer you already own**. An old desktop gathering dust is perfect. A laptop with **16 GB of RAM** (8 GB if you're patient and close everything else) works fine. You're running two virtual machines at once, so RAM is the thing that matters most.
- **Virtualization software** – pick one, both are free:
- **VirtualBox** (Oracle) – runs on any Windows, simple, great for beginners. My pick if you're new.
- **Hyper-V** – built into Windows 10/11 **Pro** (not Home). Free, fast, no download. Use this if you've got Pro.
- **Windows Server ISO** – Microsoft gives you a free **180-day evaluation**. Search "Windows Server 2022 evaluation download" (2025's out too – either is fine). Grab the ISO.
- **A Windows client ISO** – Windows 10 or 11. **Important: it must be Pro, Enterprise, or Education**. Windows **Home cannot join a domain** – that's not a bug, Microsoft just leaves the feature out. (Grab a Win10/11 eval or use a Pro install you already have.)
- **A couple of quiet evenings**. This is not a 20-minute thing. That's the point.

A note on expectations: the first time you do this, something *will* go sideways. A download will be the wrong file. A setting won't stick. A machine won't talk to the other one. **That is the homelab working as intended.** You're not here to click "Next" and feel smart. You're here to hit walls and learn to climb them. Keep going.

Build it

Concise, but real. Follow it in order. Don't skip the IP step – it's the one the whole guide hinges on.

Step 0 – Set up your virtual network first

In VirtualBox or Hyper-V, create a network the two VMs share so they can see each other but stay walled off from your home network.

- **VirtualBox:** create both VMs, then set each one's network adapter to "**Internal Network**" (same name on both, e.g. `lab-net`). Internal Network = these two machines talk to each other and nothing else. Clean and contained.
- **Hyper-V:** in Virtual Switch Manager, create a **Private** virtual switch and attach both VMs to it. Same idea.

Step 1 – Install Windows Server

1. Create a VM (4 GB RAM minimum, 60 GB disk), mount the Server ISO, boot it.
2. Run through setup. When it asks for an edition, pick "**Desktop Experience**" – that gives you the normal Windows GUI instead of a bare command line. Much friendlier while you're learning.
3. Set an Administrator password when prompted. **Write it down somewhere.** Locking yourself out of your own lab on night one is a rite of passage you can skip.
4. Boot to the desktop. Congrats, that's a server.

Step 2 – Give the server a static IP

A domain controller's address can **never** change – every client on the network finds it by that exact IP. So we nail it down by hand.

1. Open **Network & Sharing Center** → **Change adapter settings**.
2. Right-click the adapter → **Properties** → **Internet Protocol Version 4 (TCP/IPv4)** → **Properties**.
3. Choose "**Use the following IP address**" and set: - **IP address:** `192.168.10.10` - **Subnet mask:** `255.255.255.0` - **Preferred DNS server:** `127.0.0.1` (*the server points DNS at itself, because it's about to become the DNS server*)
4. OK out.

Remember `192.168.10.10`. That's your DC. The whole lab revolves around it.

Step 3 – Install the AD DS role

This is what turns a plain server into a domain controller.

1. **Server Manager** opens on boot. Click **Manage** → **Add Roles and Features**.
2. Click **Next** through to **Server Roles**. Check "**Active Directory Domain Services**." It'll prompt to add some required features – say yes.
3. Next, next, **Install**. Let it finish. **Don't close it yet** when it's done.

Step 4 – Promote it to a Domain Controller

Installing the role isn't enough – you have to *promote* the server to actually run the domain.

1. In Server Manager, a **yellow flag/notification** appears at the top. Click it → "**Promote this server to a domain controller**."
2. Choose "**Add a new forest**." (A forest is just the top-level container for your whole domain world. You're building from nothing, so you make a new one.)
3. **Root domain name:** make one up in `something.local` form – e.g. `lab.local`. This is your company's domain name.
4. Set a **DSRM (Directory Services Restore Mode) password**. Write it down too.

5. Notice that **DNS Server** is already checked by default – leave it. Your DC will be the DNS server for the whole lab. (This is *exactly* why we'll point the client's DNS here later.)
6. Click **Next** through the rest. It validates prerequisites (a DNS delegation warning is normal and safe to ignore here), then **Install**.
7. The server **reboots** on its own. When it comes back, log in as **LAB\Administrator** – see how the login is now prefixed with your domain? That means it worked. You're a domain admin of your own little kingdom.

Step 5 – Create a domain user

A real network isn't just admins. Make a normal employee account.

1. Server Manager → **Tools** → **Active Directory Users and Computers**.
2. Expand your domain (**lab.local**), right-click the **Users** folder → **New** → **User**.
3. Make someone up – first name, last name, logon name like **jdoe** . Set a password. Uncheck "User must change password at next logon" so it doesn't nag you during testing.
4. Done. **LAB\jdoe** now exists and can log into any machine that joins the domain.

Step 6 – Stand up the client and point its DNS at the DC

Now the other machine – the employee PC.

1. Create a second VM, install **Windows 10/11 Pro** (remember: **Pro/Enterprise/Education only**). Put it on the **same internal/private network** as the server.
2. This is the **most important setting in the whole build**. Give the client a static IP on the same subnet and point its DNS at the DC: - Network adapter → **IPv4** → **Properties**. - **IP address:** **192.168.10.20** - **Subnet mask:** **255.255.255.0** - **Preferred DNS server:** **192.168.10.10** ← **the DC's IP**. - **Remove any other DNS** (no **8.8.8.8** , no **1.1.1.1**). If the client asks Google where **lab.local** is, Google has no idea. Only your DC knows. So the client *must* ask the DC.

Why this matters more than anything else: a domain is found *by name* (**lab.local**), and the only machine on Earth that can translate that name into an address is your DC, because it's the DNS server. If the client's DNS doesn't point at the DC, the client is asking for directions from someone who's never heard of the place. **Wrong DNS is the #1 reason a domain join fails**. Burn that into your brain – it's about to come up.

Step 7 – Join the domain

1. On the client: **Settings** → **System** → **About** → **Rename this PC (advanced)** – or just open **Control Panel** → **System** → **Change settings** → **Change**.
2. Under "Member of," select **Domain** and type **lab.local** .
3. Click OK. It asks for credentials of an account allowed to join – use **LAB\Administrator** and the password.
4. If everything's right, you get a "**Welcome to the lab.local domain**" message. Reboot.
5. After reboot, at the login screen choose "**Other user**" and log in as **LAB\jdoe** – your domain employee, logging into a domain PC, authenticated by your domain controller.

When that works, stop and feel it for a second. You just built a working corporate network from nothing. That's not a flashcard. That's a *thing you did*.

Break it (this is the part that matters)

Here's where most guides stop. They get you to "Welcome to the domain" and call it a win. But "it worked the first time" teaches you almost nothing. **The job isn't building things that work. The job is fixing things that don't.**

So let me hand you my actual midnight.

Picture it: it's late, you're tired, and you hit **Join** – and instead of "Welcome," you get an error. *"An Active Directory Domain Controller could not be contacted."* Or it just hangs and fails. You did everything right. You're sure of it. And it won't join.

This is the moment. Don't panic, don't reinstall everything, don't start randomly flipping settings. **Diagnose it like a pro – from the bottom up.** Network problems live in layers, so you check the layers in order, lowest first. You don't guess. You *walk every step*.

Layer 1 – Can the client even reach the DC? Ping it.

On the client, open Command Prompt:

```
ping 192.168.10.10
```

- **Replies come back?** Good. The machines can physically talk. Move up to Layer 2.
- **"Request timed out" / "Destination host unreachable"?** Stop. The problem is *underneath* the domain. The two machines can't even reach each other. Don't touch domain settings yet – they're not the issue.

Layer 2 – Are they even on the same network? (This was my bug.)

Check the client's actual address:

```
ipconfig
```

Look hard at the **IPv4 Address** and **Subnet Mask**. This is where I found my midnight bug. My workstation's IP **wasn't on the right network at all**. The DC was on `192.168.10.x`, and my client had somehow ended up on a *different* subnet – a different neighborhood entirely. Same building, wrong floor. The ping failed not because the DC was down, but because the client was trying to reach an address that, from where it was sitting, didn't exist on its network.

The fix was almost insultingly simple once I saw it: **put the client's IP back on the same subnet as the DC** (`192.168.10.20` , mask `255.255.255.0`), and suddenly the ping flew, and the join worked instantly.

Hours of frustration. One wrong number. **That's IT.**

The lesson that stuck – the one I told in the interview – wasn't "set the right IP." It was *"check the most basic thing first, even when you're sure it's fine."* The IP felt too obvious to be wrong. It was wrong. **The boring, foundational layer is exactly where the bug usually hides.**

So when you break your lab – and you should *deliberately* break it, more on that in a second – check the network layer before you blame the fancy stuff.

Layer 3 – Is the client's DNS pointed at the DC?

Still in `ipconfig`, check the **DNS Servers** line. It needs to say `192.168.10.10` – your DC – and nothing else.

```
nslookup lab.local
```

- **It returns your DC's address?** DNS is good.
- **"can't find" / it answers from 8.8.8.8 ?** There's your bug. The client is asking the wrong machine where the domain is. Fix the DNS setting (Step 6), and the join goes through.

Now – break it on purpose

Here's the homework that builds the instinct. With your lab working, **go break it yourself and watch the failure:**

1. Change the client's DNS to 8.8.8.8 . Try to join. Watch it fail. *That's* what a DNS problem looks like.
2. Put the client on the wrong subnet (192.168.**99**.20). Try to ping the DC. Watch it time out. *That's* what my midnight looked like.
3. Fix each one and watch it recover.

Do this and something clicks that no flashcard ever delivers: you stop *memorizing* failures and start *recognizing* them. Six months from now, when a real user's machine won't join a real domain at a real job, your hands will already know where to look. You'll be the calm one. Because you've sat in this exact frustration at midnight – on purpose – and walked out the other side.

That's the instinct they're actually hiring for.

Talk about it

You built it. You broke it. Now you have to be able to **prove it and tell it** – because a thing you can't talk about might as well not exist on a résumé. Turn this one build into three deliverables.

1. The runbook (write it down *as you go*)

A runbook is just your own honest notes: what you built, what broke, how you fixed it. Keep a plain text or Markdown file open while you work and dump everything into it – the wins *and* the screw-ups. **The screw-ups are the valuable part.** Don't sanitize them out.

2. The GitHub README (your proof-of-work)

Make a free GitHub account. New repo: `homelab-active-directory` . Put a README in it. Now you have a public artifact you can link on your résumé and in applications – proof you don't just *say* you know AD, you *did* it.

Here's a template – fill in the brackets, drop in a couple of screenshots:

```
# Homelab: Active Directory Domain Controller + ClientJoin

## What I built
A small Windows enterprise network from scratch:
- Windows Server 2022 promoted to a Domain Controller (AD DS + DNS)
- Domain: lab.local
- A Windows 11 Pro client joined to the domain
- A domain user account (jdoe) authenticating on the client

## Environment
- Host: [your PC], [Hyper-V / VirtualBox]
- DC: 192.168.10.10 (static), DNS pointed at itself
- Client: 192.168.10.20 (static), DNS pointed at the DC

## What broke (and what I learned)
The client failed to join the domain. I diagnosed it bottom-up:
1. Pinged the DC – failed.
2. Ran ipconfig – found the client's IP was on the wrong subnet,
   so it couldn't reach the DC at all.
3. Corrected the IP to the DC's subnet – ping succeeded, join succeeded.

**Takeaway:** check the foundational network layer first. The most
"obvious" setting is often the broken one.

## Screenshots
[ AD Users and Computers ] [ successful domain join ] [ jdoe logged in ]
```

That README, honestly, is more convincing than another line of certs. It shows you *think* like the job.

3. The 30-second story (rehearse it out loud)

This is the one that gets you hired. When they ask *"tell me about a time you fixed something,"* you don't freeze – you tell **this**. Out loud. A few times. Until it's smooth:

"In my homelab I built a Windows domain controller and a client that joins it. One night the client just would not join the domain – kept failing. Instead of guessing, I worked it bottom-up: pinged the DC, it failed. Ran ipconfig and found the client's IP was on the wrong subnet – it literally couldn't reach the domain controller. I corrected the IP, the ping went through, and the join worked instantly. The lesson that stuck with me is to check the most basic network layer first, even when you're sure it's fine – that's usually where the problem's hiding."

That's it. Specific. Honest. Walks every step. Ends on a lesson. **You're not reciting trivia – you're telling them you've already been the person who fixes the thing.** That's what I did with barely any cert finished, and it worked.

What's next

Real talk: **this guide is complete on its own.** If you build this lab, break it, document it, and rehearse the story, you have a genuine GitHub piece and a genuine interview answer. That's a real step forward and I'm not going to pretend you need to buy anything to make it count. You don't.

But if this clicked for you – if breaking things on purpose and walking out the other side felt *good* – there's more trail.

The [\\$39 Lab Manual](#) is where I take this method and go deep. More builds (DHCP, Group Policy, file shares with real permissions, a second site), and more importantly, more **deliberate breaks** – a whole catalog of "now here's what it looks like when *this* fails, and here's how a pro walks it back." Every build comes with its own runbook template and its own interview story, so you finish with a *portfolio*, not just one repo. It's the thing I wish someone had sold me for forty bucks when I was studying at midnight, instead of me piecing it together the hard way.

No pressure, no countdown timer, no hype. If the Starter got you hired, tell me – that genuinely makes my week. And if you want to go further, the Manual's right here when you do.

Now go build it. Then break it. Then come tell somebody about it.

– **James** *Twilight Tech*

Sources used to verify the technical steps

- [Promote a server to a domain controller – ManageEngine](#)
- [AD DS Configuration Wizard page descriptions – Microsoft Learn](#)
- [Join a computer to a domain – Microsoft Learn](#)
- [How to Join a Windows 10 PC to a Domain – groovyPost](#)

BUILD IT. BREAK IT. TALK ABOUT IT.

TWILIGHT//TECH · twilighttech.io · from cert to career